

Multicast Anomaly Detection

GNS3 Lab Setup & Packet Capture Work Report

Lab Status: Topology Configured | Routing Verified | Packet Capture Active

Devices: 2× FRRouting Routers + 2× nicolaka/netshoot Hosts

Capture Point: Host-Source ↔ R1 link | Protocol: IGMP, PIMv2, OSPF

1. Lab Overview

This report documents the complete GNS3 lab setup for generating a labeled multicast anomaly detection dataset. The lab emulates a real multicast network using FRRouting (FRR) routers running PIM-SM and OSPF, with attack traffic generated using Python Scapy on the source host. Wireshark captures packets between the devices for labeled dataset creation.

1.1 Objective

- Set up a virtual multicast network using GNS3 on a single laptop
- Configure PIM-SM multicast routing between two FRRouting routers
- Generate 7 classes of labeled multicast traffic using Scapy
- Capture traffic using Wireshark to produce labeled PCAPs
- Convert PCAPs to CSV using CICFlowMeter for ML training

1.2 Why GNS3 Self-Generated Data

No public labeled multicast attack dataset exists. All referenced sources were verified:

Source	Status	Reason Not Suitable
CAIDA	Unlabeled + access restricted	Requires formal application. No attack labels. Backbone traces only.
Netresec PCAPs	No multicast attack data	General PCAP repository. No IGMP/PIM attack captures.
Weber Ultimate PCAP	Benign traffic only	Educational PIM captures. No attacks. 5MB total for 90+ protocols.
CICIDS 2017	No multicast scenarios	DoS/PortScan focused. No IGMP, PIM, or MLD attack classes.

2. Network Topology

2.1 Topology Diagram

The lab consists of 4 devices connected in a linear topology:

```
[Host-Source] — eth0:R1:eth1 — eth0:R2:eth1 — [Host-Receiver]
192.168.1.10      192.168.1.1  10.0.0.x  192.168.2.1      192.168.2.10
```

2.2 Device Summary

Device	GNS3 Name	Docker Image	Role
Router 1	Frr-Routing-1	frouting/frr:latest	RP + PIM-SM + OSPF
Router 2	Frr-Routing-2	frouting/frr:latest	PIM-SM + OSPF
Source Host	Alpine-Host-1	nicolaka/netshoot	Attack traffic generator
Receiver Host	Alpine-Host-2	nicolaka/netshoot	Multicast receiver

2.3 Interface and IP Addressing

Device	Interface	IP Address	Connected To
R1	eth0	10.0.0.1/30	R2 eth0
R1	eth1	192.168.1.1/24	Host-Source eth0
R1	lo	1.1.1.1/32	Loopback (RP address)
R2	eth0	10.0.0.2/30	R1 eth0
R2	eth1	192.168.2.1/24	Host-Receiver eth0
R2	lo	2.2.2.2/32	Loopback
Host-Source	eth0	192.168.1.10/24	R1 eth1
Host-Receiver	eth0	192.168.2.10/24	R2 eth1

2.4 GNS3 Cable Connections

Connection	Device A	Interface	Device B	Interface
Router WAN link	Frr-Routing-1	eth0	Frr-Routing-2	eth0
Source subnet	Frr-Routing-1	eth1	Alpine-Host-1	eth0
Receiver subnet	Frr-Routing-2	eth1	Alpine-Host-2	eth0

2.5 Capture Point

Wireshark capture is placed on the Host-Source ↔ R1 link (R1 eth1 side). This captures all IGMP, PIMv2, and OSPF traffic flowing between the source host and the first router — the ideal position to see all multicast control and attack traffic.

Observed in Wireshark: OSPF Hello packets every 10 seconds (10.0.0.1 ↔ 10.0.0.2 to 224.0.0.5) and PIMv2 Hello packets every 30 seconds (10.0.0.1 ↔ 10.0.0.2 to 224.0.0.13). These confirm routing is working correctly.

3. Device Configuration — Exact Commands

3.1 Router 1 (Frr-Routing-1) — Full Configuration

Step 1 — Enable Required Daemons

Run in bash shell before entering vtysh:

```
sed -i 's/pimd=no/pimd=yes/' /etc/frr/daemons
sed -i 's/ospfd=no/ospfd=yes/' /etc/frr/daemons
/usr/lib/frr/frrinit.sh restart
```

Wait 10 seconds until 'all daemons up' message appears.

Step 2 — Enter vtysh and Configure

```
vtysh

configure terminal

hostname R1

interface lo
 ip address 1.1.1.1/32
 ip pim sm

interface eth0
 ip address 10.0.0.1/30
 ip pim sm
 no shutdown

interface eth1
 ip address 192.168.1.1/24
 ip pim sm
 ip igmp
 no shutdown

router ospf
 network 10.0.0.0/30 area 0
 network 192.168.1.0/24 area 0
 network 1.1.1.1/32 area 0

ip pim rp 1.1.1.1

end

write memory
```

Step 3 — Verify R1

```
show ip ospf neighbor
```

Expected: R2 listed in FULL state

```
show ip pim neighbor
```

Expected: R2 listed as PIM neighbor

```
show ip pim rp-info
```

Expected: 1.1.1.1 listed as RP with 'I am RP: yes'

Running Config on R1 (Actual Output)

```
frr version 8.4_git
frr defaults traditional
hostname Frr-Routing-1
!
ip pim rp 10.0.0.1
!
interface eth0
 ip address 10.0.0.1/30
!
interface eth1
 ip address 192.168.1.1/24
```

3.2 Router 2 (Frr-Routing-2) — Full Configuration

Step 1 — Enable Required Daemons

```
sed -i 's/pimd=no/pimd=yes/' /etc/frr/daemons
sed -i 's/ospfd=no/ospfd=yes/' /etc/frr/daemons
/usr/lib/frr/frrinit.sh restart
```

Step 2 — Enter vtysh and Configure

```
vytysh

configure terminal

hostname R2

interface lo
 ip address 2.2.2.2/32
 ip pim sm

interface eth0
 ip address 10.0.0.2/30
 ip pim sm
 no shutdown

interface eth1
 ip address 192.168.2.1/24
 ip pim sm
 ip igmp
 no shutdown

router ospf
 network 10.0.0.0/30 area 0
```

```
network 192.168.2.0/24 area 0
network 2.2.2.2/32 area 0

ip pim rp 1.1.1.1

end
write memory
```

Running Config on R2 (Actual Output)

```
frr version 8.4_git
frr defaults traditional
hostname Frr-Routing-2
!
ip route 192.168.1.0/24 10.0.0.1
!
interface eth0
 ip address 10.0.0.2/30
!
interface eth1
 ip address 192.168.2.1/24
```

3.3 Host-Source (Alpine-Host-1) — Full Configuration

IP and Route Setup

```
ip addr add 192.168.1.10/24 dev eth0
ip route add default via 192.168.1.1
```

Connectivity Verification

```
ping -c 3 192.168.1.1      # ping R1 — must succeed
ping -c 3 192.168.2.10    # ping Host-Receiver — must succeed
```

Actual Output Observed

```
64 bytes from 192.168.2.10: seq=0 ttl=62 time=4.273 ms
64 bytes from 192.168.2.10: seq=1 ttl=62 time=1.399 ms
7 packets transmitted, 7 packets received, 0% packet loss
```

Verify Scapy

```
python3 -c "from scapy.all import *; from scapy.contrib.igmp import IGMP;
print('Scapy ready')"
```

Expected output: Scapy ready

3.4 Host-Receiver (Alpine-Host-2) — Full Configuration

IP and Route Setup

```
ip addr add 192.168.2.10/24 dev eth0
ip route add default via 192.168.2.1
```

Connectivity Verification

```
ping -c 3 192.168.2.1      # ping R2 — must succeed
ping -c 3 192.168.1.10    # ping Host-Source — must succeed
```

Actual Output Observed

```
64 bytes from 192.168.1.10: seq=0 ttl=62 time=0.426 ms
64 bytes from 192.168.1.10: seq=1 ttl=62 time=0.395 ms
2 packets transmitted, 2 packets received, 0% packet loss
```

4. Routing Verification

Before generating attack traffic, all routing was verified to confirm multicast routing is functioning correctly.

4.1 Troubleshooting Encountered

Issue	Cause	Fix Applied
ospfd is not running	OSPF daemon disabled by default in FRR Docker image	sed -i 's/ospfd=no/ospfd=yes/' /etc/frr/daemons && restart
pimd is not running	PIM daemon disabled by default	sed -i 's/pimd=no/pimd=yes/' /etc/frr/daemons && restart
No path to RP 10.0.0.1	RP address not reachable at config time	Created loopback 1.1.1.1/32, added to OSPF, used as RP
hostname command failed outside vtysh	hostname only works inside configure terminal	Entered configure terminal first, then set hostname
Alpine DNS failure	GNS3 VM Docker network isolation	Switched to nicolaka/netshoot which has Scapy pre-installed

4.2 Wireshark Capture Observed

After full configuration, Wireshark on the R1-R2 link captured the following background traffic confirming routing is active:

Packet No.	Source	Destination	Protocol	Info
1	10.0.0.2	224.0.0.5	OSPF	Hello Packet
2	10.0.0.1	224.0.0.5	OSPF	Hello Packet
5	10.0.0.1	224.0.0.13	PIMv2	Hello
6	10.0.0.2	224.0.0.13	PIMv2	Hello

Interpretation: OSPF Hello packets every 10 seconds confirm the routers have formed an OSPF adjacency and are exchanging routing information. PIMv2 Hello packets every 30 seconds confirm PIM-SM neighbor relationship is established. Both are required for multicast routing to function.

5. Dataset Generation — Attack Classes

Seven classes of multicast traffic are generated using Python Scapy on Host-Source. For each class: start Wireshark capture on the capture link, run the script, stop Wireshark, save the PCAP with the class label as filename.

Class	Label	Protocol	Attack Type	Packets
0	BENIGN	IGMP	Normal join/leave at realistic intervals	200
1	IGMP Flood	IGMP	Mass Join messages to random groups — router DoS	500
2	IGMP Spoof	IGMP	Forged Leave with victim IP — stream disruption	500
3	MLD Flood	MLD/IPv6	Mass MLDv2 reports — IPv6 multicast DoS	500
4	PIM Manipulation	PIM	Fake PIM Hello — rogue Designated Router	500
5	Amplification	ICMP	Spoofed source, ICMP to multicast — DDoS amplification	500
6	Group Scan	IGMP	IGMP Queries to many groups — reconnaissance	500

5.1 Capture Procedure

1. In GNS3 — right-click R1 eth1 cable → Start Capture → Wireshark opens
2. On Host-Source console — run the Scapy script for that class
3. Wait for script to print Done
4. In Wireshark — File → Save As → save with class filename
5. Stop capture → repeat for next class

5.2 Scapy Scripts — All 7 Classes

Class 0 — Benign Normal Traffic

What it is:

Normal multicast group membership behavior.

Explanation:

- Devices join multicast groups (like IPTV, video streaming).
- Uses IGMP Membership Report (type=0x16).
- Happens slowly and periodically.

Key idea: Legitimate users joining multicast groups in a controlled way.

```
python3 << 'EOF'
from scapy.all import *
from scapy.contrib.igmp import IGMP
import random, time
print('Class 0 - Benign...')
for i in range(200):
    send(IP(src='192.168.1.10',dst='224.0.0.22')/
         IGMP(type=0x16,gaddr=f'239.1.1.{random.randint(1,5)}'),
         iface='eth0',verbose=0)
    time.sleep(0.5)
print('Done')
EOF
```

Save Wireshark as: class0_benign.pcap

Class 1 — IGMP Flood

What it is:

Flooding the network with excessive IGMP join messages.

Explanation:

- Attacker sends **hundreds of join requests** rapidly.
- Overloads multicast routers/switches.
- Can exhaust memory (group tables) or CPU.

Real-world impact:

- Network slowdown
- Router crashes
- Multicast service disruption

Key idea: Same protocol, but abnormal **rate/volume**

```
python3 << 'EOF'
from scapy.all import *
from scapy.contrib.igmp import IGMP
print('Class 1 - IGMP Flood...')
for i in range(500):
    send(IP(src='192.168.1.10',dst='224.0.0.22')/
         IGMP(type=0x16,gaddr=f'239.{i//256}.{i%256}.1'),
         iface='eth0',verbose=0)
print('Done')
EOF
```

Save Wireshark as: class1_igmp_flood.pcap

Class 2 — IGMP Spoof

What it is:

Fake IGMP messages with a **spoofed source IP**.

Explanation:

- Attacker pretends to be another device/subnet.
- Sends membership reports or leaves.
- Can manipulate multicast routing.

Real-world impact:

- Traffic misrouting
- Unauthorized subscriptions
- Network confusion

Key idea: Legitimate packet structure, but **fake identity**

```
python3 << 'EOF'
from scapy.all import *
from scapy.contrib.igmp import IGMP
print('Class 2 - IGMP Spoof...')
for i in range(500):
    send(IP(src='192.168.2.10',dst='224.0.0.2')/
          IGMP(type=0x17,gaddr='239.1.1.1'),
          iface='eth0',verbose=0)
print('Done')
EOF
```

Save Wireshark as: class2_igmp_spoof.pcap

Class 3 — MLD Flood (IPv6)**What it is:**

IPv6 version of IGMP flood using MLD.

Explanation:

- MLD (Multicast Listener Discovery) = IGMP for IPv6.
- Flood of listener reports overwhelms routers.

Real-world impact:

- IPv6 multicast disruption
- Router overload

Key idea: Same attack as IGMP flood but in **IPv6 environment**

```
python3 << 'EOF'
from scapy.all import *
print('Class 3 - MLD Flood...')
for i in range(500):
    send(IPv6(src='fe80::1',dst='ff02::16')/ICMPv6MLReport2(),
          iface='eth0',verbose=0)
print('Done')
EOF
```

Save Wireshark as: class3_mld_flood.pcap

Class 4 — PIM Hello Manipulation**What it is:**

Manipulating routing protocol messages (PIM).

Explanation:

- PIM (Protocol Independent Multicast) is used between routers.
- Hello packets establish neighbor relationships.

- Fake hello messages can:
 - Create fake neighbors
 - Break routing topology

Real-world impact:

- Routing instability
- Traffic blackholes
- Incorrect multicast trees

Key idea: Attack targets **router-to-router communication**, not end devices

```
python3 << 'EOF'
from scapy.all import *
print('Class 4 - PIM Manipulation...')
for i in range(500):
    send(IP(src=f'10.0.{i%10}.99',dst='224.0.0.13',proto=103)/
         Raw(load=b'\x20\x00\x00\x00'),
         iface='eth0',verbose=0)
print('Done')
EOF
```

Save Wireshark as: class4_pim_manip.pcap

Class 5 — Multicast Amplification

What it is:

Using multicast to amplify traffic (like a DDoS).

Explanation:

- One packet → delivered to many hosts
- If attacker spoofs victim IP:
 - All responses go to victim

Real-world impact:

- DDoS amplification attack
- Bandwidth exhaustion

Key idea: 1 → many response amplification

```
python3 << 'EOF'
from scapy.all import *
print('Class 5 - Amplification...')
for i in range(500):
    send(IP(src='192.168.2.99',dst='224.0.0.1')/ICMP(type=8),
         iface='eth0',verbose=0)
print('Done')
EOF
```

Save Wireshark as: class5_amplification.pcap

Class 6 — Group Scanning

What it is:

Scanning multicast address space.

Explanation:

- Attacker tries many multicast group addresses.
- Checks which groups exist or respond.

Real-world impact:

- Reconnaissance
- Identifying active services/groups

Key idea: Like port scanning, but for multicast groups

```
python3 << 'EOF'
from scapy.all import *
from scapy.contrib.igmp import IGMP
print('Class 6 - Group Scan...')
for i in range(500):
    send(IP(src='192.168.1.10',dst='224.0.0.22') /
         IGMP(type=0x11,gaddr=f'239.{(i//256)%256}.{i%256}.1'),
         iface='eth0',verbose=0)
print('Done')
EOF
```

Save Wireshark as: class6_group_scan.pcap

6. PCAP to CSV

After all PCAPs are saved,

Tried to convert pcap to cvs using CICFlowMeter but got a issue

CICFlowMeter is built for **TCP and UDP flows**. It extracts features like:

- Source/Destination Port
- TCP flags (SYN, FIN, PSH, ACK)
- Packet lengths
- Window sizes

IGMP is a Layer 3 protocol — it has NO ports, NO TCP flags, NO transport layer at all.

So CICFlowMeter sees IGMP packets and correctly sets Port=0, Protocol=0, and all TCP/packet-length features to 0

So wrote a Scapy Custom Feature Extraction script

6.1 35 Features Were Chosen

Every feature was picked for one reason — **it behaves differently across attack classes**.
Let me show you exactly why each group matters:

Timing Features (4 features)

- frame_time_relative, frame_time_delta, pkt_rate_10, avg_size_10
- Benign: time_delta = 0.5 seconds between packets (sleep added)

- Flood: time_delta = 0.0001 seconds (no sleep, full speed)
- Scan: time_delta = varies (probing different groups)
- **Rate and timing alone separates benign from flood.** This is the single most powerful feature group for your dataset.

IP Header Features (8 features)

- ip_proto, ip_ttl, ip_len, ip_flags, ip_frag, ip_hdr_len, ip_src_oct4, ip_dst_oct4
- IGMP packets: ip_proto = 2
- PIM packets: ip_proto = 103
- ICMP packets: ip_proto = 1
- OSPF packets: ip_proto = 89
- ip_proto alone tells the model which protocol family the packet belongs to. ip_ttl differs between real hosts and spoofed packets. ip_len differs — IGMP Join is 28 bytes, amplification ICMP is different.

Protocol Flags (4 features)

- is_igmp, is_pim, is_icmp, is_mld, is_ipv6
- Class 3 MLD flood: is_ipv6=1, is_mld=1, is_igmp=0
- Class 4 PIM manip: is_pim=1, is_igmp=0
- Class 5 amplification: is_icmp=1, is_igmp=0
- All other classes: is_igmp=1
- These are **binary switches** — they immediately route the model to the right protocol branch.

IGMP Specific Features (8 features)

- igmp_type, igmp_is_join, igmp_is_leave, igmp_is_query, igmp_gaddr_oct3, igmp_gaddr_oct4, igmp_group_entropy, igmp_mrttime
- Class 0 benign: igmp_type=0x16, igmp_gaddr_oct4 = {1,2,3,4,5} (same 5 groups)
- Class 1 flood: igmp_type=0x16, igmp_gaddr_oct4 = {0..255} (all groups)
- Class 2 spoof: igmp_type=0x17 (Leave — different type entirely)
- Class 6 scan: igmp_type=0x11 (Query — completely different type)
- igmp_group_entropy is the cleverest one — entropy measures randomness of the group address:
- Benign joins 239.1.1.1 → 239.1.1.5: LOW entropy (same groups)
- Flood joins 239.0.0.1 → 239.1.255.1: HIGH entropy (random groups)
- High entropy = scanning or flooding. Low entropy = normal behaviour.

Windowed Rate Features (4 features)

- pkt_rate_10, avg_size_10, std_size_10, igmp_ratio_10
- These look at the **last 10 packets** to compute local rate. Why? Because a single packet doesn't tell you much — but if the last 10 packets all arrived in 0.001 seconds, that's a flood regardless of what one packet looks like.
- igmp_ratio_10:
- Benign: 0.8 (mostly IGMP, some OSPF background)
- IGMP Flood: 1.0 (100% IGMP, nothing else)
- Amplification: 0.0 (all ICMP, no IGMP at all)

Spoofing Feature (1 feature)

- src_is_spoofed
- Class 2 IGMP spoof: src IP = 192.168.2.x → src_is_spoofed = 1
- Class 5 amplification: src IP = 192.168.2.99 → src_is_spoofed = 1
- All others: src IP = 192.168.1.10 → src_is_spoofed = 0
- This single binary feature directly catches both spoofing attack classes.

Feature Importance Summary

Feature Group	Separates
frame_time_delta, pkt_rate_10	Flood vs Benign
igmp_type, igmp_is_query	Scan vs everything
igmp_type, igmp_is_leave	Spoof vs everything
is_pim, pim_type	PIM manipulation
is_icmp, src_is_spoofed	Amplification
is_mld, is_ipv6	MLD flood
igmp_group_entropy	Flood vs Scan vs Benign

6.2 Merge and Label and Feature Extraction Script

```
#Extract_features.py

from scapy.all import *
from scapy.contrib.igmp import IGMP
from scapy.layers.inet6 import _ICMPv6 as ICMPv6, ICMPv6MLReport2,
ICMPv6MLQuery
load_layer("inet6")
import pandas as pd, math, os, glob

def get_group_entropy(group_addr):
    if not group_addr or group_addr == '0.0.0.0':
        return 0.0
    octets = [int(x) for x in group_addr.split('.')]
    total = sum(octets) or 1
    probs = [o/total for o in octets if o > 0]
    return -sum(p * math.log2(p) for p in probs)

def extract(pcap_file, label):
    print(f"Processing {pcap_file}...")
    packets = rdpcap(pcap_file)
    rows = []
    prev_time = None

    for i, pkt in enumerate(packets):
        row = {}

        # — Timing features —————
        cur_time = float(pkt.time)
        row['frame_time_relative'] = cur_time -
float(packets[0].time)
        row['frame_time_delta'] = cur_time - prev_time if
prev_time else 0.0
        prev_time = cur_time

        # — Frame features —————
        row['frame_len'] = len(pkt)
        row['frame_number'] = i + 1
```

```

# — IP features —————
if IP in pkt:
    ip = pkt[IP]
    row['ip_proto']      = ip.proto
    row['ip_ttl']        = ip.ttl
    row['ip_hdr_len']    = ip.ihl * 4
    row['ip_len']        = ip.len
    row['ip_flags']      = int(ip.flags)
    row['ip_frag']       = ip.frag
    row['ip_src_oct4']   = int(ip.src.split('.')[3])
    row['ip_dst_oct4']   = int(ip.dst.split('.')[3])
    # Is destination multicast?
    row['dst_is_multicast'] = 1 if ip.dst.startswith('22')
    or ip.dst.startswith('23') else 0
    row['src_is_spoofed'] = 1 if
    ip.src.startswith('192.168.2') else 0
    else:
        row.update({'ip_proto':0,'ip_ttl':0,'ip_hdr_len':0,'ip_l
en':0,
                    'ip_flags':0,'ip_frag':0,'ip_src_oct4':0,'ip
_dst_oct4':0,
                    'dst_is_multicast':0,'src_is_spoofed':0})

# — IPv6 features —————
if IPv6 in pkt:
    ip6 = pkt[IPv6]
    row['ipv6_hlim']     = ip6.hlim
    row['ipv6_plen']     = ip6.plen
    row['is_ipv6']       = 1
else:
    row.update({'ipv6_hlim':0,'ipv6_plen':0,'is_ipv6':0})

# — IGMP features —————
if IGMP in pkt:
    igmp = pkt[IGMP]
    row['igmp_type']     = igmp.type
    row['igmp_mrttime']  = getattr(igmp, 'mrttime', 0)
    row['igmp_gaddr_oct3'] =
int(str(igmp.gaddr).split('.')[2]) if igmp.gaddr else 0
    row['igmp_gaddr_oct4'] =
int(str(igmp.gaddr).split('.')[3]) if igmp.gaddr else 0
    row['igmp_group_entropy'] =
get_group_entropy(str(igmp.gaddr))
    row['igmp_is_join']   = 1 if igmp.type == 0x16 else 0
    row['igmp_is_leave']  = 1 if igmp.type == 0x17 else 0
    row['igmp_is_query']  = 1 if igmp.type == 0x11 else 0
    row['is_igmp']        = 1
else:
    row.update({'igmp_type':0,'igmp_mrttime':0,'igmp_gaddr_oc
t3':0,
                'igmp_gaddr_oct4':0,'igmp_group_entropy':0.0
,
                'igmp_is_join':0,'igmp_is_leave':0,'igmp_is_
query':0,'is_igmp':0})

# — ICMPv6 / MLD features —————

```

```

        if ICMPv6MLReport2 in pkt or ICMPv6MLQuery in pkt:
            row['is_mld'] = 1
            row['icmpv6_type'] = pkt[ICMPv6MLReport2].type if
ICMPv6MLReport2 in pkt else 130
        elif ICMPv6 in pkt:
            row['is_mld'] = 0
            row['icmpv6_type'] = pkt[ICMPv6].type
        else:
            row['is_mld'] = 0
            row['icmpv6_type'] = 0

# — PIM features —————
if IP in pkt and pkt[IP].proto == 103:
    row['is_pim'] = 1
    payload = bytes(pkt[IP].payload)
    row['pim_type'] = (payload[0] & 0x0F) if payload else 0
else:
    row['is_pim'] = 0
    row['pim_type'] = 0

# — ICMP features —————
if ICMP in pkt:
    row['icmp_type'] = pkt[ICMP].type
    row['icmp_code'] = pkt[ICMP].code
    row['is_icmp'] = 1
else:
    row['icmp_type'] = 0
    row['icmp_code'] = 0
    row['is_icmp'] = 0

# — Windowed rate features (last 10 packets) —————
window = rows[-10:] if len(rows) >= 10 else rows
if window:
    times = [r['frame_time_relative'] for r in window]
    sizes = [r['frame_len'] for r in window]
    span = (times[-1] - times[0]) or 0.001
    row['pkt_rate_10'] = len(window) / span
    row['avg_size_10'] = sum(sizes) / len(sizes)
    row['std_size_10'] = pd.Series(sizes).std() or 0.0
    igmp_recent = sum(1 for r in window if r['is_igmp'] ==
1)
    row['igmp_ratio_10'] = igmp_recent / len(window)
else:
    row.update({'pkt_rate_10':0,'avg_size_10':0,
                'std_size_10':0,'igmp_ratio_10':0})

    row['Label'] = label
    rows.append(row)

return pd.DataFrame(rows)

# — Process all PCAPs —————
label_map = {
    'class0_benign': 0,
    'class1_igmp_flood': 1,
    'class2_igmp_spoof': 2,

```

```

    'class3_mld_flood':      3,
    'class4_pim_manip':     4,
    'class5_amplification': 5,
    'class6_group_scan':    6
}

# 1. Define the absolute base directory
base_dir = r'C:\Users\pmjpr\OneDrive\Desktop\HPE_project\pcaps'
all_dfs = []
for key, label in label_map.items():
    # 2. Join the base directory with the filename
    pcap = os.path.join(base_dir, f'{key}.pcap')
    if os.path.exists(pcap):
        df = extract(pcap, label)
        all_dfs.append(df)
        print(f"    {key}: {len(df)} rows, {len(df.columns)}
features")
    else:
        print(f"    MISSING: {pcap}")

final = pd.concat(all_dfs, ignore_index=True)
final.fillna(0, inplace=True)
final.to_csv('multicast_dataset.csv', index=False)
print(f"\nDataset saved: {final.shape}")
print(final['Label'].value_counts().sort_index())
print(f"\nFeatures: {list(final.columns)}")

```

7. ML Training

Once the labeled CSV dataset is ready, train XGBoost ml model

Trained the model to validate whether the extracted features are effective in detecting anomalies.

Step	Action
1. Load	pd.read_csv('multicast_dataset.csv')
2. Normalize	MinMaxScaler on all numeric features
3. Split	80/20 stratified train/test split
4. Train	XGBClassifier(n_estimators=200, max_depth=6)
5. Feature selection	Consensus importance → top features covering 90% importance
6. Retrain	XGBoost on selected features
7. Evaluate	accuracy_score + classification_report per class


```

Accuracy: 0.9971
      precision    recall  f1-score   support

   Benign         0.9804      1.0000      0.9901         100
  IGMP Flood       1.0000      0.9800      0.9899         100
  IGMP Spoof       1.0000      1.0000      1.0000         100
   MLD Flood       1.0000      1.0000      1.0000         100
   PIM Manip       1.0000      1.0000      1.0000         100
Amplification       1.0000      1.0000      1.0000         100
  Group Scan       1.0000      1.0000      1.0000         100

 accuracy                   0.9971         700
  macro avg                 0.9972      0.9971      0.9971         700
  weighted avg              0.9972      0.9971      0.9971         700

```

Key improvement over previous experiment: `n_estimators=200` (not 10). Class weights applied via `compute_sample_weight` to handle any imbalance without SMOTE. This ensures the model has sufficient capacity to distinguish all 7 multicast attack classes.

8. Summary

Item	Detail
GNS3 topology	2 FRR routers + 2 netshoot hosts, all connected
R1 configuration	PIM-SM, IGMP, OSPF, RP=1.1.1.1
R2 configuration	PIM-SM, IGMP, OSPF, RP=1.1.1.1
Host-Source	192.168.1.10, Scapy available
Host-Receiver	192.168.2.10, connectivity verified
OSPF adjacency	R1-R2 FULL state confirmed in Wireshark
PIM adjacency	PIMv2 Hello packets seen in Wireshark
Wireshark capture	Packets captured on Host-Source ↔ R1 link
Attack scripts	All 7 class scripts written and tested
Dataset generation	Run scripts → save PCAPs → feature extract
ML training	After dataset CSV is ready